

Tutorial: unix

Victor Eijkhout*

Unix is an *Operating System (OS)*, that is, a layer of software between the user or a user program and the hardware. It takes care of files and screen output, and it makes sure that many processes can exist side by side on one system. However, it is not immediately visible to the user.

Most of this tutorial will work on any Unix-like platform, however, there is not just one Unix:

- Traditionally there are a few major flavors of Unix: *ATT* or *System V*, and *BSD*. Apple has Darwin which is close to BSD; IBM and HP have their own versions of Unix, and Linux is yet another variant. These days many Unix versions adhere to the *POSIX* standard. The differences between these are deep down and if you are taking this tutorial you probably won't see them for quite a while.
- Within Linux there are various *Linux distributions* such as *Red Hat* or *Ubuntu*. These mainly differ in the organization of system files and again you probably need not worry about them.
- The issue of command shells will be discussed below. This actually forms the most visible difference between different computers 'running Unix'.

1 Shells

Most of the time that you use Unix, you are typing commands which are executed by an interpreter called the *shell*. The shell makes the actual OS calls. There are a few possible Unix shells available

- Most of this tutorial is focused on the *sh* or *bash* shell.
- For a variety of reasons (see for instance section ??), *bash*-like shells are to be preferred over the *csh* or *tcsh* shell. These latter ones will not be covered in this tutorial.
- Recent versions of the *Apple Mac OS* have the *zsh* as default. While this shell has many things in common with *bash*, we will point out differences explicitly.

2 Files and such

Purpose. In this section you will learn about the Unix file system, which consists of *directories* that store *files*. You will learn about *executable* files and commands for displaying data files.

* This tutorial is taken from the book 'Introduction to High Performance Scientific Computing', <http://tinyurl.com/EijkhoutHPC>.

2.1 Looking at files

Purpose. In this section you will learn commands for displaying file contents.

Commands learned in this section	
<code>ls</code>	list files or directories
<code>touch</code>	create new/empty file or update existing file
<code>cat > filename</code>	enter text into file
<code>cp</code>	copy files
<code>mv</code>	rename files
<code>rm</code>	remove files
<code>file</code>	report the type of file
<code>cat filename</code>	display file
<code>head,tail</code>	display part of a file
<code>less,more</code>	incrementally display a file

2.1.1 `ls`

Without any argument, the `ls` command gives you a listing of files that are in your present location.

Exercise 3.1. Type `ls`. Does anything show up?

Intended outcome. If there are files in your directory, they will be listed; if there are none, no output will be given. This is standard Unix behavior: no output does not mean that something went wrong, it only means that there is nothing to report.

Exercise 3.2. If the `ls` command shows that there are files, do `ls name` on one of those. By using an option, for instance `ls -s name` you can get more information about `name`.

Things to watch out for. If you mistype a name, or specify a name of a non-existing file, you'll get an error message.

The `ls` command can give you all sorts of information. In addition to the above `ls -s` for the size, there is `ls -l` for the 'long' listing. It shows (things we will get to later such as) ownership and permissions, as well as the size and creation date.

Remark *There are several dates associated with a file, corresponding to changes in content, changes in permissions, and access of any sort. The `stat` command gives all of them.*

2.1.2 `cat`

The `cat` command (short for 'concatenate') is often used to display files, but it can also be used to create some simple content.

Exercise 3.3. Type `cat > newfilename` (where you can pick any filename) and type some text.

Conclude with `Control-d` on a line by itself: press the `Control` key and hold it while you press the `d` key. Now use `cat` to view the contents of that file: `cat newfilename`.

Intended outcome. In the first use of `cat`, text was appended from the terminal to a file; in the second the file was cat'ed to the terminal output. You should see on your screen precisely what you typed into the file.

Things to watch out for. Be sure to type Control-d as the first thing on the last line of input. If you really get stuck, Control-c will usually get you out. Try this: start creating a file with `cat > filename` and hit Control-c in the middle of a line. What are the contents of your file?

Remark *Instead of Control-d you will often see the notation ^D. The capital letter is for historic reasons: you use the control key and the lowercase letter.*

2.1.3 `man`

The primary (though not always the most easily understood) source for unix commands is the `man` command, for 'manual'. The descriptions available this way are referred to as the *manual pages*.

Exercise 3.4. Read the man page of the `ls` command: `man ls`. Find out the size and the time / date of the last change to some files, for instance the file you just created.

Intended outcome. Did you find the `ls -s` and `ls -l` options? The first one lists the size of each file, usually in kilobytes, the other gives all sorts of information about a file, including things you will learn about later.

The `man` command puts you in a mode where you can view long text documents. This viewer is common on Unix systems (it is available as the `more` or `less` system command), so memorize the following ways of navigating: Use the space bar to go forward and the u key to go back up. Use g to go to the beginning of the text, and G for the end. Use q to exit the viewer. If you really get stuck, Control-c will get you out.

Remark *If you already know what command you're looking for, you can use `man` to get online information about it. If you forget the name of a command, `man -k keyword` can help you find it.*

2.1.4 `touch`

The `touch` command creates an empty file, or updates the timestamp of a file if it already exists. Use `ls -l` to confirm this behavior.

2.1.5 `cp`, `mv`, `rm`, `ln`

The `cp` can be used for copying a file (or directories, see below): `cp file1 file2` makes a copy of `file1` and names it `file2`.

Exercise 3.5. Use `cp file1 file2` to copy a file. Confirm that the two files have the same contents. If you change the original, does anything happen to the copy?

Intended outcome. You should see that the copy does not change if the original changes or is deleted.

Things to watch out for. If `file2` already exists, you will get an error message.

A file can be renamed with `mv`, for 'move'.

Exercise 3.6. Rename a file. What happens if the target name already exists?

Files are deleted with `rm`. This command is dangerous: there is no undo. For this reason you can do `rm -i` (for ‘interactive’) which asks your confirmation for every file. See section 2.4 for more aggressive removing.

Sometimes you want to refer to a file from two locations. This is not the same as having a copy: you want to be able to edit either one, and have the other one change too. This can be done with `ln`: ‘link’.

This snippet creates a file and a link to it:

```
$ echo contents > arose
$ cd mydir
$ ln ../arose anyothername
$ cat anyothername
contents
$ echo morestuff >> anyothername
$ cd ..
$ cat arose
contents
morestuff
```

2.1.6 head, tail, file

There are more commands for displaying a file, parts of a file, or information about a file.

Exercise 3.7. Do `ls /usr/share/words` or `ls /usr/share/dict/words` to confirm that a file with words exists on your system. Now experiment with the commands `head`, `tail`, `more`, and `wc` using that file.

Intended outcome. `head` displays the first couple of lines of a file, `tail` the last, and `more` uses the same viewer that is used for man pages. Read the man pages for these commands and experiment with increasing and decreasing the amount of output. The `wc` (‘word count’) command reports the number of words, characters, and lines in a file.

Remark The `tail` command can display the end of a file ‘as it grows’: do `tail -F some_log_file`.

Another useful command is `file`: it tells you what type of file you are dealing with.

Exercise 3.8. Do `file foo` for various ‘foo’: a text file, a directory, or the `/bin/ls` command.

Intended outcome. Some of the information may not be intelligible to you, but the words to look out for are ‘text’, ‘directory’, or ‘executable’.

At this point it is advisable to learn to use a text *editor*, such as `emacs` or `vi`.

2.2 Directories

Purpose. Here you will learn about the Unix directory tree, how to manipulate it and how to move around in it.

Commands learned in this section

<code>ls</code>	list the contents of directories
<code>mkdir</code>	make new directory
<code>cd</code>	change directory
<code>pwd</code>	display present working directory

A unix file system is a tree of directories, where a directory is a container for files or more directories. We will display directories as follows:

```

/..... The root of the directory tree
├ bin ..... Binary programs
└ home ..... Location of users directories

```

The root of the Unix directory tree is indicated with a slash. Do `ls /` to see what the files and directories there are in the root. Note that the root is not the location where you start when you reboot your personal machine, or when you log in to a server.

Exercise 3.9. The command to find out your current working directory is `pwd`. Your home directory is your working directory immediately when you log in. Find out your home directory.

Intended outcome. You will typically see something like `/home/yourname` or `/Users/yourname`. This is system dependent.

Do `ls` to see the contents of the working directory. In the displays in this section, directory names will be followed by a slash: `dir/` but this character is not part of their name. You can get this output by using `ls -F`, and you can tell your shell to use this output consistently by stating `alias ls='ls -F'` at the start of your session. Example:

```

/home/you/
├ adirectory/
└ afile

```

The command for making a new directory is `mkdir`.

Exercise 3.10. Make a new directory with `mkdir newdir` and view the current directory with `ls`.

Intended outcome. You should see this structure:

```

/home/you/
├ newdir/.....the new directory

```

If you need to create a directory several levels deep, you could

```

mkdir sub1
cd sub1
mkdir sub2
cd sub2
## et cetera

```

but it's shorter to use the `-p` option (for 'parent') and write:

```

mkdir -p sub1/sub2/sub3

```

which creates any needed intermediate levels. The `-p` option can also be used if the directory already exists. (What happens normally if you try to `mkdir` a directory that already exists?)

The command for going into another directory, that is, making it your working directory, is `cd` ('change directory'). It can be used in the following ways:

- `cd` Without any arguments, `cd` takes you to your home directory.
- `cd <absolute path>` An absolute path starts at the root of the directory tree, that is, starts with `/`. The `cd` command takes you to that location.
- `cd <relative path>` A relative path is one that does not start at the root. This form of the `cd` command takes you to `<yourcurrentdir>/<relative path>`.

Exercise 3.11. Do `cd newdir` and find out where you are in the directory tree with `pwd`. Confirm with `ls` that the directory is empty. How would you get to this location using an absolute path?

Intended outcome. `pwd` should tell you `/home/you/newdir`, and `ls` then has no output, meaning there is nothing to list. The absolute path is `/home/you/newdir`.

Exercise 3.12. Let's quickly create a file in this directory: `touch onefile`, and another directory: `mkdir otherdir`. Do `ls` and confirm that there are a new file and directory.

Intended outcome. You should now have:

```
/home/you/
├─ newdir/.....you are here
│   ├─ onefile
│   └─ otherdir/
```

The `ls` command has a very useful option: with `ls -a` you see your regular files and hidden files, which have a name that starts with a dot. Doing `ls -a` in your new directory should tell you that there are the following files:

```
/home/you/
├─ newdir/.....you are here
│   ├─ .
│   ├─ ..
│   ├─ onefile
│   └─ otherdir/
```

The single dot is the current directory, and the double dot is the directory one level back.

Exercise 3.13. Predict where you will be after `cd ./otherdir/..` and check to see if you were right.

Intended outcome. The single dot sends you to the current directory, so that does not change anything. The `otherdir` part makes that subdirectory your current working directory. Finally, `..` goes one level back. In other words, this command puts your right back where you started.

Since your home directory is a special place, there are shortcuts for `cd`'ing to it: `cd` without arguments, `cd ~`, and `cd $HOME` all get you back to your home.

Go to your home directory, and from there do `ls newdir` to check the contents of the first directory you created, without having to go there.

Exercise 3.14. What does `ls ..` do?

Intended outcome. Recall that `..` denotes the directory one level up in the tree: you should see your own home directory, plus the directories of any other users.

Let's practice the use of the single and double dot directory shortcuts.

Exercise 3.15. From your home directory:

```
mkdir -p sub1/sub2/sub3
cd sub1/sub2/sub3
touch a
```

You now have a file `sub1/sub2/sub3/a`

1. How do you move it to `sub1/sub2/a`?
2. Go: `cd sub1/sub2`
How do you now move the file to `sub1/a`?
3. Go to your home directory: `cd`
How do you move `sub1/a` to here?

Exercise 3.16. Can you use `ls` to see the contents of someone else's home directory? In the previous exercise you saw whether other users exist on your system. If so, do `ls ../thatotheruser`.

Intended outcome. If this is your private computer, you can probably view the contents of the other user's directory. If this is a university computer or so, the other directory may very well be protected – permissions are discussed in the next section – and you get `ls: ../otheruser: Permission denied`.

Make an attempt to move into someone else's home directory with `cd`. Does it work?

You can make copies of a directory with `cp`, but you need to add a flag to indicate that you recursively copy the contents: `cp -r`. Make another directory `somedir` in your home so that you have

```
/home/you/
├─ newdir/ ..... you have been working in this one
└─ somedir/ ..... you just created this one
```

What is the difference between `cp -r newdir somedir` where `somedir` is an existing directory, and `cp -r newdir thirddir` where `thirddir` is not an existing directory?

2.3 Permissions

Purpose. In this section you will learn about how to give various users on your system permission to do (or not to do) various things with your files.

Unix files, including directories, have permissions, indicating 'who can do what with this file'. Actions that can be performed on a file fall into three categories:

- reading `r`: any access to a file (displaying, getting information on it) that does not change the file;

- writing `w`: access to a file that changes its content, or even its metadata such as ‘date modified’;
- executing `x`: if the file is executable, to run it; if it is a directory, to enter it.

The people who can potentially access a file are divided into three classes too:

- the user `u`: the person owning the file;
- the group `g`: a group of users to which the owner belongs;
- other `o`: everyone else.

(For more on groups and ownership, see section 13.3.)

The nine permissions are rendered in sequence

user	group	other
rwX	rwX	rwX

For instance `rw-r--r--` means that the owner can read and write a file, the owner’s group and everyone else can only read.

Permissions are also rendered numerically in groups of three bits, by letting `r = 4`, `w = 2`, `x = 1`:

rwX
421

Common codes are `7 = rwX` and `6 = rw`. You will find many files that have permissions `755` which stands for an executable that everyone can run, but only the owner can change, or `644` which stands for a data file that everyone can see but again only the owner can alter. You can set permissions by the `chmod` command:

```
chmod <permissions> file          # just one file
chmod -R <permissions> directory # directory, recursively
```

Examples:

```
chmod 766 file # set to rwxrw-rw-
chmod g+w file # give group write permission
chmod g=rx file # set group permissions
chod o-w file # take away write permission from others
chmod o= file # take away all permissions from others.
chmod g+r,o-x file # give group read permission
                  # remove other execute permission
```

The man page gives all options.

Exercise 3.17. Make a file `foo` and do `chmod u-r foo`. Can you now inspect its contents? Make the file readable again, this time using a numeric code. Now make the file readable to your classmates. Check by having one of them read the contents.

Intended outcome. 1. A file is only accessible by others if the surrounding folder is readable. Can you figure out how to do this? 2. When you’ve made the file ‘unreadable’ by yourself, you can still `ls` it, but not `cat` it: that will give a ‘permission denied’ message.

Make a file `com` with the following contents:

```
#!/bin/sh
echo "Hello world!"
```


This is a legitimate shell script. What happens when you type `./com`? Can you get the script executed?

In the three permission categories it is clear who ‘you’ and ‘others’ refer to. How about ‘group’? We’ll go into that in section 13.

Exercise 3.18. Suppose you’re an instructor and you want to make a ‘dropbox’ directory for students to deposit homework assignments in. What would be an appropriate mode for that directory? (Assume that you have co-teachers that are in your group, and who also need to be able to see the contents. In other words, group permission should be identical to the owner permission.)

Remark *There are more obscure permissions. For instance the `setuid` bit declares that the program should run with the permissions of the creator, rather than the user executing it. This is useful for system utilities such `passwd` or `mkdir`, which alter the password file and the directory structure, for which root privileges are needed. Thanks to the `setuid` bit, a user can run these programs, which are then so designed that a user can only make changes to their own password entry, and their own directories, respectively. The `setuid` bit is set with `chmod: chmod 4ugo file`.*

2.4 Wildcards

You already saw that `ls filename` gives you information about that one file, and `ls` gives you all files in the current directory. To see files with certain conditions on their names, the *wildcard* mechanism exists. The following wildcards exist:

*	any number of characters
?	any character.

Example:

```
%% ls
s      sk      ski      skiing skill
%% ls ski*
ski      skiing skill
```

The second option lists all files whose name start with `ski`, followed by any number of other characters’; below you will see that in different contexts `ski*` means ‘`sk` followed by any number of `i` characters’’. Confusing, but that’s the way it is.

You can use `rm` with wildcards, but this can be dangerous.

```
rm -f foo      ## remove foo if it exists
rm -r foo      ## remove directory foo with everything in it
rm -rf foo/*   ## delete all contents of foo
```

Zsh note. Removing with a wildcard `rm foo*` is an error if there are no such files. Set `setopt +o nomatch` to allow no matches to occur.

3 Text searching and regular expressions

Purpose. In this section you will learn how to search for text in files.

For this section you need at least one file that contains some amount of text. You can for instance get random text from <http://www.lipsum.com/feed/html>.

The `grep` command can be used to search for a text expression in a file.

Exercise 3.19. Search for the letter `q` in your text file with `grep q yourfile` and search for it in all files in your directory with `grep q *`. Try some other searches.

Intended outcome. In the first case, you get a listing of all lines that contain a `q`; in the second case, `grep` also reports what file name the match was found in: `qfile: this line has q in it`.

Things to watch out for. If the string you are looking for does not occur, `grep` will simply not output anything. Remember that this is standard behavior for Unix commands if there is nothing to report.

In addition to searching for literal strings, you can look for more general expressions.

<code>^</code>	the beginning of the line
<code>\$</code>	the end of the line
<code>.</code>	any character
<code>*</code>	any number of repetitions
<code>[xyz]</code>	any of the characters <code>xyz</code>

This looks like the wildcard mechanism you just saw (section 2.4) but it's subtly different. Compare the example above with:

```
%% cat s
sk
ski
skill
skiing
%% grep "ski*" s
sk
ski
skill
skiing
```

In the second case you search for a string consisting of `sk` and any number of `i` characters, including zero of them.

Some more examples: you can find

- All lines that contain the letter 'q' with `grep q yourfile`;
- All lines that start with an 'a' with `grep "^a" yourfile` (if your search string contains special characters, it is a good idea to use quote marks to enclose it);
- All lines that end with a digit with `grep "[0-9]$" yourfile`.

Exercise 3.20. Construct the search strings for finding

- lines that start with an uppercase character, and
- lines that contain exactly one character.

Intended outcome. For the first, use the range characters `[]`, for the second use the period to match any character.

Exercise 3.21. Add a few lines `x = 1`, `x = 2`, `x = 3` (that is, have different numbers of spaces between `x` and the equals sign) to your test file, and make `grep` commands to search for all assignments to `x`.

The characters in the table above have special meanings. If you want to search that actual character, you have to *escape* it.

Exercise 3.22. Make a test file that has both `abc` and `a.c` in it, on separate lines. Try the commands `grep "a.c" file`, `grep a\.c file`, `grep "a\.c" file`.

Intended outcome. You will see that the period needs to be escaped, and the search string needs to be quoted. In the absence of either, you will see that `grep` also finds the `abc` string.

3.1 Line manipulation with `cut` and `tr`

Two useful utilities for ‘editing’ a line are `tr` and `cut`.

First `tr` which stands for ‘translate’. It takes two characters or ranges of characters: the first character, or any character in the first range, is translated to the second, or the corresponding character in the second range.

Obvious examples are translating between upper and lower case:

```
# translate upper case to lower
echo MyString | tr A-Z a-z
```

Exercise 3.23. Let `a` and `b` be strings. How do you test if they are equal, up to differences in case? Can you do this as a single command? If you know how to write shell scripts, write one that accepts two arguments and return true if they are equal up to case differences.

You can also use `tr` to delete, rather than translate, characters:

```
#remove whitespace: spaces and tabs
echo "foo bar" | tr -d " \t"
```

Another tool for editing lines is `cut`, which will cut up a line and display certain parts of it. For instance, `cut -c 2-5 myfile`

will display the characters in position 2–5 of every line of `myfile`. Make a test file and verify this example.

Maybe more useful, you can give `cut` a delimiter character and have it split a line on occurrences of that delimiter. For instance, your system will mostly likely have a file `/etc/passwd` that contains user information¹, with every line consisting of fields separated by colons. For instance:

```
daemon*:1:1:System Services:/var/root:/usr/bin/false
nobody*:2:-2:Unprivileged User:/var/empty:/usr/bin/false
root*:0:0:System Administrator:/var/root:/bin/sh
```

The seventh and last field is the login shell of the user; `/bin/false` indicates that the user is unable to log in.

You can display users and their login shells with:

1. This is traditionally the case; on Mac OS information about users is kept elsewhere and this file only contains system services.

```
cut -d ":" -f 1,7 /etc/passwd
```

This tells cut to use the colon as delimiter, and to print fields 1 and 7.

4 Other useful commands: tar

The `tar` command stands for ‘tape archive’, that is, it was originally meant to package files on a tape. (The ‘archive’ part derives from the `ar` command.) These days, it’s used to package files together for distribution on web sites and such: if you want to publish a library of hundreds of files this bundles them into a single file.

The two most common options are for

1. creating a tar file:

```
tar fc package.tar directory_with_stuff
```

pronounced ‘tar file create’, and

2. unpacking a tar file:

```
tar fx package.tar
# this creates the directory that was packaged
```

pronounced ‘tar file extract’.

Text files can often be compressed to a large extent, so adding the `z` compression for `gzip` is a good idea:

```
tar fcz package.tar.gz directory_with_stuff
tar fx package.tar.gz
```

Naming the ‘gzipped’ file `package.tgz` is also common.

5 Command execution

If you type something in the shell, you are actually asking the underlying interpreter to execute a command. Some commands are built-ins, others can be names of programs that are stored somewhere, in a system location or in your own account. This section will go into the mechanisms.

Remark Like any good programming language, the shell language as comments. Any line starting with a hash character `#` is ignored. In Apple’s `zsh`, the comment character is disabled. Do

```
setopt interactivecomments
```

to enable it.

5.1 Search paths

Commands learned in this section

<i>which</i>	location of executable command
<i>type</i>	description of commands, functions, ...

Purpose. In this section you will learn how Unix determines what to do when you type a command name.

If you type a command such as `ls`, the shell does not just rely on a list of commands: it will actually go searching for a program by the name `ls`. This means that you can have multiple different commands with the same name, and which one gets executed depends on which one is found first.

Exercise 3.24. What you may think of as ‘Unix commands’ are often just executable files in a system directory. Do *which* `ls`, and do an `ls -l` on the result.

Intended outcome. The location of `ls` is something like `/bin/ls`. If you `ls` that, you will see that it is probably owned by root. Its executable bits are probably set for all users.

The locations where unix searches for commands is the *search path*, which is stored in the *environment variable* (for more details see below) `PATH`.

Exercise 3.25. Do `echo $PATH`. Can you find the location of `cd`? Are there other commands in the same location? Is the current directory ‘.’ in the path? If not, do `export PATH=".: $PATH"`. Now create an executable file `cd` in the current director (see above for the basics), and do `cd`.

Intended outcome. The path will be a list of colon-separated directories, for instance `/usr/bin:/usr/local/bin:/usr/X11R6/bin`. If the working directory is in the path, it will probably be at the end: `/usr/X11R6/bin:.` but most likely it will not be there. If you put ‘.’ at the start of the path, unix will find the local `cd` command before the system one.

Some people consider having the working directory in the path a security risk. If your directory is writable, someone could put a malicious script named `cd` (or any other system command) in your directory, and you would execute it unwittingly.

The safest way to execute a program in the current directory is:

```
./my_program
```

This holds both for compiled programs and shell scripts; section 9.2.

Remark Not all Unix commands correspond to executables. The `type` command gives more information than *which*:

```
$ type echo
echo is a shell builtin
$ type \ls
ls is an alias for ls -F
$ unalias ls
$ type ls
```

```
ls is /bin/ls
$ type module
module is a shell function from /usr/local/Cellar/lmod/8.7.2/init/zsh
```

5.2 Aliases

It is possible to define your own commands as aliases of existing commands.

Exercise 3.26. Do `alias chdir=cd` and convince yourself that now `chdir` works just like `cd`. Do `alias rm='rm -i'`; look up the meaning of this in the man pages. Some people find this alias a good idea; can you see why?

Intended outcome. The `-i` ‘interactive’ option for `rm` makes the command ask for confirmation before each delete. Since unix does not have a trashcan that needs to be emptied explicitly (as on Windows or the Mac OS), this can be a good idea.

5.3 Command sequencing

There are various ways of having multiple commands on a single commandline.

5.3.1 Simple sequencing

First of all, you can type

```
command1 ; command2
```

This is convenient if you repeat the same two commands a number of times: you only need to up-arrow once to repeat them both.

There is a problem: if you type

```
cc -o myprog myprog.c ; ./myprog
```

and the compilation fails, the program will still be executed, using an old version of the executable if that exists. This is very confusing.

A better way is:

```
cc -o myprog myprog.c && ./myprog
```

which only executes the second command if the first one was successful.

5.3.2 Pipelining

Instead of taking input from a file, or sending output to a file, it is possible to connect two commands together, so that the second takes the output of the first as input. The syntax for this is `cmdone | cmdtwo`; this is called a pipeline. For instance, `grep a yourfile | grep b` finds all lines that contains both an `a` and a `b`.

Exercise 3.27. Construct a pipeline that counts how many lines there are in your file that contain the string `th`. Use the `wc` command (see above) to do the counting.

5.3.3 Backquoting

There are a few more ways to combine commands. Suppose you want to present the result of `wc` a bit nicely. Type the following command

```
echo The line count is wc -l foo
```

where `foo` is the name of an existing file. The way to get the actual line count echoed is by the *backquote*:

```
echo The line count is `wc -l foo`
```

Anything in between backquotes is executed before the rest of the command line is evaluated.

Exercise 3.28. The way `wc` is used here, it prints the file name. Can you find a way to prevent that from happening?

There is another mechanism for out-of-order evaluation:

```
echo "There are $( cat Makefile | wc -l ) lines"
```

This mechanism makes it possible to nest commands, but for compatibility and legacy purposes backquotes may still be preferable when nesting is not needed.

5.3.4 Grouping in a subshell

Suppose you want to apply output redirection to a couple of commands in a row:

```
configure ; make ; make install > installation.log 2>&1
```

This only catches the last command. You could for instance group the three commands in a subshell and catch the output of that:

```
( configure ; make ; make install ) > installation.log 2>&1
```

5.4 Exit status

Commands can fail. If you type a single command on the command line, you see the error, and you act accordingly when you type the next command. When that failing command happens in a script, you have to tell the script how to act accordingly. For this, you use the *exit status* of the command: this is a value (zero for success, nonzero otherwise) that is stored in an internal variable, and that you can access with `$?` .

Example. Suppose we have a directory that is not writable

```
[testing] ls -ld nowrite/
dr-xr-xr-x 2 eijkhout 506 68 May 19 12:32 nowrite/
```

and write try to create a file there:

```
[testing] cd nowrite/
[nowrite] cat ../newfile
#!/bin/bash
touch $1
```

```

echo "Created file: $1"
[nowrite] ../newfile myfile
touch: myfile: Permission denied
Created file: myfile
[nowrite] ls
[nowrite]

```

The script reports that the file was created even though it wasn't.

Improved script:

```

[nowrite] cat ../betterfile
#!/bin/bash
touch $1
if [ $? -eq 0 ] ; then
    echo "Created file: $1"
else
    echo "Problem creating file: $1"
fi

[nowrite] ../betterfile myfile
touch: myfile: Permission denied
Problem creating file: myfile

```

Exercise 3.29. Referring to section 6.4, how can you prevent the error message from *touch* to show on your terminal?

5.5 Processes and jobs

Commands learned in this section

<code>ps</code>	list (all) processes
<code>kill</code>	kill a process
<code>CTRL-C</code>	kill the foreground job
<code>CTRL-Z</code>	suspect the foreground job
<code>jobs</code>	give the status of all jobs
<code>fg</code>	bring the last suspended job to the foreground
<code>fg %3</code>	bring a specific job to the foreground
<code>bg</code>	run the last suspended job in the background

The Unix operating system can run many programs at the same time, by rotating through the list and giving each only a fraction of a second to run each time. The command `ps` can tell you everything that is currently running.

Exercise 3.30. Type `ps`. How many programs are currently running? By default `ps` gives you only programs that you explicitly started. Do `ps guwax` for a detailed list of everything that is running. How many programs are running? How many belong to the root user, how many to you?

Intended outcome. To count the programs belonging to a user, pipe the `ps` command through an appropriate `grep`, which can then be piped to `wc`.

In this long listing of `ps`, the second column contains the *process numbers*. Sometimes it is useful to have those: if a program misbehaves you can `kill` it with

```
kill 123456
```

where 12345 is the process number.

The `cut` command explained above can cut certain position from a line: type `ps guwax | cut -c 10-14`.

To get dynamic information about all running processes, use the `top` command. Read the man page to find out how to sort the output by CPU usage.

Processes that are started in a shell are known as *jobsjob (unix)*. In addition to the process number, they have a job number. We will now explore manipulating jobs.

When you type a command and hit return, that command becomes, for the duration of its run, the *foreground process*. Everything else that is running at the same time is a *background process*.

Make an executable file `hello` with the following contents:

```
#!/bin/sh
while [ 1 ] ; do
    sleep 2
    date
done
```

and type `./hello`.

Exercise 3.31. Type `Control-z`. This suspends the foreground process. It will give you a number like `[1]` or `[2]` indicating that it is the first or second program that has been suspended or put in the background. Now type `bg` to put this process in the background. Confirm that there is no foreground process by hitting return, and doing an `ls`.

Intended outcome. After you put a process in the background, the terminal is available again to accept foreground commands. If you hit return, you should see the command prompt. However, the background process still keeps generating output.

Exercise 3.32. Type `jobs` to see the processes in the current session. If the process you just put in the background was number 1, type `fg %1`. Confirm that it is a foreground process again.

Intended outcome. If a shell is executing a program in the foreground, it will not accept command input, so hitting return should only produce blank lines.

Exercise 3.33. When you have made the `hello` script a foreground process again, you can kill it with `Control-c`. Try this. Start the script up again, this time as `./hello &` which immediately puts it in the background. You should also get output along the lines of `[1] 12345` which tells you that it is the first job you put in the background, and that 12345 is its process ID. Kill the script with `kill %1`. Start it up again, and kill it by using the process number.

Intended outcome. The command `kill 12345` using the process number is usually enough to kill a running program. Sometimes it is necessary to use `kill -9 12345`.

5.6 Shell customization

Above it was mentioned that `ls -F` is an easy way to see which files are regular, executable, or directories; by typing `alias ls='ls -F'` the `ls` command will automatically expanded to `ls -F` every time it is invoked. If you would like this behavior in every login session, you can add the `alias` command to your `.profile` file. Other shells than `sh/bash` have other files for such customizations.

6 Input/output Redirection

Purpose. In this section you will learn how to feed one command into another, and how to connect commands to input and output files.

So far, the unix commands you have used have taken their input from your keyboard, or from a file named on the command line; their output went to your screen. There are other possibilities for providing input from a file, or for storing the output in a file.

6.1 Standard files

Unix has three standard files that handle input and output:

Standard file	
<code>stdin</code> (number 0)	is the file that provides input for processes.
<code>stdout</code> (number 1)	is the file where the output of a process is written.
<code>stderr</code> (number 2)	is the file where error output is written.

In an interactive session, all three files are connected to the user terminal. Using input or output redirection then means that the input is taken or the output sent to a different file than the terminal.

6.2 Input redirection

The `grep` command had two arguments, the second being a file name. You can also write `grep string < yourfile`, where the less-than sign means that the input will come from the named file, `yourfile`. This is known as *input redirection*.

6.3 Output redirection

Just as with the input, you can redirect the output of your program. In the simplest case, `grep string yourfile > outfile` will take what normally goes to the terminal, and *redirect* the output to `outfile`. The output file is created if it didn't already exist, otherwise it is overwritten. (To append, use `grep text yourfile >> outfile`.)

Exercise 3.34. Take one of the `grep` commands from the previous section, and send its output to a file. Check that the contents of the file are identical to what appeared on your screen before. Search for a string that does not appear in the file and send the output to a file. What does this mean for the output file?

Intended outcome. Searching for a string that does not occur in a file gives no terminal output. If you redirect the output of this `grep` to a file, it gives a zero size file. Check this with `ls` and `wc`.

Exercise 3.35. Generate a text file that contains your information:

```
My user name is:
eijkhout
My home directory is:
/users/eijkhout
I made this script on:
isp.tacc.utexas.edu
```

where you use the commands `whoami`, `pwd`, `hostname`.

Bonus points if you can get the ‘prompt’ and output on the same line.

Hint: see section 5.3.3.

Sometimes you want to run a program, but ignore the output. For that, you can redirect your output to the system *null device*: `/dev/null`.

```
yourprogram >/dev/null
```

Here are some useful idioms:

6.4 Error redirection

The `stdout` and `stderr` files are treated very similar by default: they both go to the terminal output. And normally that is fine, but suppose you run a program that generates errors and pipe it to `less`:

```
./myprogram | less
```

If you scroll back you’ll see that you can not revisit the error messages. Both `stdout` and `stderr` went to your screen, but only `stdout` went to `less`.

To manipulate `stderr` you need to use the stream number ‘1’ for `stdout`, and ‘2’ for `stderr`. With this, you can redirect the two streams separately. For instance,

```
./myprogram 1>my.out 2>my.err
```

sends them to two separate files.

If you want to ignore errors, send them to `/dev/null`, which is writable file that is not actually stored. So:

```
./myprogram 2>/dev/null
```

In the first example of this section having separate treatment was not the solution, but rather the problem. To send both streams to a pipe, use

```
./myprogram 2>&1 | less
```

If you want to send them both to the same file, you need the somewhat counterintuitive sequencing:

```
./myprogram 1>/dev/null 2>&1
```

Idiom

program 2>/dev/null	redirect only errors to the null device
program >/dev/null 2>&1	redirect output to dev-null, and errors to output
program 2>&1 less	pipe output and errors to less

6.5 tee

What if you want to send output to a file, but also view it on your screen as it's being generated? For that, use `tee`. (The name comes from 'T-joins where one tube – hooked up to the vertical leg of the 'T' – can be connected to two others, hooked up to the left and right horizontal bar.) So:

```
./myprogram | tee my.out
```

sends output to `my.out` and displays it on screen.

Exercise 3.36.

- How would you send output to a file and to `less`?
- How would you send `stdout` and `stderr` both to `tee`?

7 Shell environment variables

Above you encountered `PATH`, which is an example of an shell, or environment, variable. These are variables that are known to the shell and that can be used by all programs run by the shell. While `PATH` is a built-in variable, you can also define your own variables, and use those in shell scripting.

Shell variables are roughly divided in the following categories:

- Variables that are specific to the shell, such as `HOME` or `PATH`.
- Variables that are specific to some program, such as `TEXINPUTS` for `TEX/LATEX`.
- Variables that you define yourself; see next.
- Variables that are defined by control structures such as `for`; see below.

You can see the full list of all variables known to the shell by typing `env`. **Remark** *This does not include variables you define yourself, unless you `export` them; see below.*

Exercise 3.37. Check on the value of the `PATH` variable by typing `echo $PATH`. Also find the value of `PATH` by piping `env` through `grep`.

We start by exploring the use of this dollar sign in relation to shell variables.

7.1 Use of shell variables

You can get the value of a shell variable by prefixing it with a dollar sign. Type the following and inspect the output:

```

echo x
echo $x
x=5
echo x
echo $x

```

You see that the shell treats everything as a string, unless you explicitly tell it to take the value of a variable, by putting a dollar in front of the name. A variable that has not been previously defined will print as a blank string.

Shell variables can be set in a number of ways. The simplest is by an assignment as in other programming languages.

When you do the next exercise, it is good to bear in mind that the shell is a text based language.

Exercise 3.38. Type `a=5` on the commandline. Check on its value with the `echo` command.

Define the variable `b` to another integer. Check on its value.

Now explore the values of `a+b` and `$a+$b`, both by `echo`'ing them, or by first assigning them.

Intended outcome. The shell does not perform integer addition here: instead you get a string with a plus-sign in it. (You will see how to do arithmetic on variables in section 10.5.)

Things to watch out for. Beware not to have space around the equals sign; also be sure to use the dollar sign to print the value.

7.2 Exporting variables

A variable set this way will be known to all subsequent commands you issue in this shell, but not to commands in new shells you start up. For that you need the `export` command. Reproduce the following session (the square brackets form the command prompt):

```

$ a=20
$ echo $a
20
$ /bin/bash
$ echo $a

$ exit
exit
$ export a=21
$ /bin/bash
$ echo $a
21
$ exit

```

You can also temporarily set a variable. Replay this scenario:

1. Find an environment variable that does not have a value:

```

$ echo $b

$

```

2. Write a short shell script to print this variable:

```
$ cat > echob
#!/bin/bash
echo $b
```

and of course make it executable: `chmod +x echob`.

3. Now call the script, preceding it with a setting of the variable `b`:

```
$ b=5 ./echob
5
```

The syntax where you set the value, as a prefix without using a separate command, sets the value just for that one command.

4. Show that the variable is still undefined:

```
$ echo $b

$
```

That is, you defined the variable just for the execution of a single command.

In section 8 you will see that the `for` construct also defines a variable; section 9.2 shows some more built-in variables that apply in shell scripts.

If you want to un-set an environment variable, there is the `unset` command.

8 Control structures

Like any good programming system, the shell has some control structures. Their syntax takes a bit of getting used to. (Different shells have different syntax; in this tutorial we only discuss the bash shell.)

8.1 Conditionals

The *conditional* of the bash shell is predictably called `if`, and it can be written over several lines:

```
if [ $PATH = "" ] ; then
    echo "Error: path is empty"
fi
```

or on a single line:

```
if [ `wc -l file` -gt 100 ] ; then echo "file too long" ; fi
```

(The backquote is explained in section 5.3.3.) There are a number of tests defined, for instance `-f somefile` tests for the existence of a file. Change your script so that it will report `-1` if the file does not exist.

The syntax of this is finicky:

- `if` and `elif` are followed by a conditional, followed by a semicolon.
- The brackets of the conditional need to have spaces surrounding them.
- There is no semicolon after `then` or `else`: they are immediately followed by some command.

Exercise 3.39. Bash conditionals have an `elif` keyword. Can you predict the error you get from this:

```
if [ something ] ; then
    foo
else if [ something_else ] ; then
    bar
fi
```

Code it out and see if you were right.

Modern versions of Bash have an extended conditional syntax with double square brackets. For instance, pattern matching:

```
if [[ $myvar == *substring* ]] ; then ....
```

Here is a short summary of possible tests, given through examples:

```
if [ $x = "foo" ]           ## text equality
if [ $x -eq 3 ]             ## numerical equality; also '-gt' &c
if [ ! -z "${x}" ]         ## test if variable is not null
if [ ! -z "${x}" -a -z "${y}" ] ## 'and' of two tests, also '-o'
if [ -d foo ]               ## is there a 'foo' directory
if [[ -x foo ]]             ## is 'foo' executable
if [[ $myvar == *substring* ]] ## pattern matching
```

8.2 Looping

In addition to conditionals, the shell has loops. A `for` loop looks like

```
for var in listofitems ; do
    something with $var
done
```

This does the following:

- for each item in `listofitems`, the variable `var` is set to the item, and
- the loop body is executed.

As a simple example:

```
for x in a b c ; do echo $x ; done
a
b
c
```

In a more meaningful example, here is how you would make backups of all your `.c` files:

```
for cfile in *.c ; do
    cp $cfile $cfile.bak
done
```

The above construct loops over words, such as the output of `ls`. To do a numeric loop, use the command `seq`:

```
$ seq 1 5
1
2
3
4
5
```

Looping over a sequence of numbers then typically looks like

```
for i in `seq 1 ${HOWMANY}` ; do echo $i ; done
```

Note the *backtick*, which is necessary to have the `seq` command executed before evaluating the loop; see section 5.3.3.

Exercise 3.40. Take a program that reads one number from standard in, and run it in a loop in the shell, using the number 10 through 20 as input.

You can break out of a loop with `break`; this can even have a numeric argument indicating how many levels of loop to break out of.

9 Scripting

The unix shells are also programming environments. You will learn more about this aspect of unix in this section.

9.1 Sourcing

You can think of a script as putting single statements together in a file so that you can repeatedly execute them. The simplest Unix mechanism to do this bundling goes like this:

1. Make a file `my_commands.txt` with any Unix commands;
2. Then


```
source my_commands.txt
# or:
. my_commands.txt
```

The effect will be as if you had typed the commands straight at the command prompt.

While this can be useful, it can also be confusing since the commands file can do `cd` or make other changes to your environment. Usually it's better to let a script run in its own sheltered world – to be precise: in a subshell

9.2 How to execute scripts

It is possible to write programs of unix shell commands. First you need to know how to put a program in a file and have it be executed. Make a file `script1` containing the following two lines:

```
#!/bin/bash
echo "hello world"
```


and type `./script1` on the command line. Result? Make the file executable and try again.

Zsh note. If you use the `zsh`, but you have `bash` scripts that you wrote in the past, they will keep working. The ‘hash-bang’ line determines which shell executes the script, and it is perfectly possible to have `bash` in your script, while using `zsh` for interactive use.

In order write scripts that you want to invoke from anywhere, people typically put them in a directory `bin` in their home directory. You would then add this directory to your *search path*, contained in `PATH`; see section 5.1.

9.3 Script arguments

You can invoke a shell script with options and arguments:

```
./my_script -a file1 -t -x file2 file3
```

You will now learn how to incorporate this functionality in your scripts.

First of all, all commandline arguments and options are available as variables `$1`, `$2` et cetera in the script, and the number of command line arguments is available as `$#`:

```
#!/bin/bash
echo "The first argument is $1"
echo "There were $# arguments in all"
```

Formally:

variable	meaning
<code>\$#</code>	number of arguments
<code>\$0</code>	the name of the script (but see below)
<code>\$1, \$2, ...</code>	the arguments
<code>\$*, \$@</code>	the list of all arguments

Exercise 3.41. Write a script that takes as input a file name argument, and reports how many lines are in that file.

Edit your script to test whether the file has less than 10 lines (use the `foo -lt bar` test), and if it does, `cat` the file. Hint: you need to use backquotes inside the test.

Add a test to your script so that it will give a helpful message if you call it without any arguments.

The standard way to parse argument is using the `shift` command, which pops the first argument off the list of arguments. Parsing the arguments in sequence then involves looking at `$1`, shifting, and looking at the new `$1`.

Code:

```
// shell/arguments.sh
while [ $# -gt 0 ]; do
    echo "argument: $1"
    shift
done
```

Output:

```
[code/shell] arguments:

./arguments.sh the quick "brown
↪fox" jumps
argument: the
argument: quick
argument: brown fox
argument: jumps
```

The variables `$@` and `$*` have a different behavior with respect to double quotes. Let's say we evaluate `myscript "1 2" 3`, then

- Using `$*` is the list of arguments after removing quotes: `myscript 1 2 3`.
- Using `"$@"` is the list of arguments, with quotes removed, in quotes: `myscript "1 2 3"`.
- Using `"$@"` preserves quotes: `myscript "1 2" 3`.

If you want to write your own commandline parsing, so that you could write

```
./myscript.sh -a -b arg
```

another useful command is `shift`. If you call this, the internal list of arguments is modified:

1. the first argument is removed, and
2. the argument count is lowered.

Supposing you have a script with some (optional) flags and one mandatory argument:

```
Usage: myscript.sh [ -a ] [ -b arg ] filename
```

you would then write:

```
while [ $# -gt 1 ] ; do
    if [ $1 = "-a" ] ; then
        ## "-a" is a a flag
        hasa=1 && shift
    elif [ $1 = "-b" ] ; then
        ## "-b" has an argument:
        ## 1. remove the -b flag
        shift
        ## 2. save the argument and remove
        bargument=$1 && shift
    else
        echo "Unknown option: $1" && exit 1
    fi
done
```

Exercise 3.42. Write a script `say.sh` that prints its text argument. However, if you invoke it with

```
./say.sh -n 7 "Hello world"
```

it should be print it as many times as you indicated. Using the option `-u`:

```
./say.sh -u -n 7 "Goodbye cruel world"
```

should print the message in uppercase. Make sure that the order of the arguments does not matter, and give an error message for any unrecognized option.

Remark Above you saw that `$0` is the name of the script being executed. This only holds scripts that have the execute bit set, and that are executed by being found on the path. If you source `./script.sh`, that variable will actually report `'bash'`. If you want the script name in such cases, use `BASH_SOURCE`:

- `${BASH_SOURCE[0]}` is the name of the script being called or sourced;
- `${BASH_SOURCE[1]}` is the name of the script calling or source that script;
- et cetera.

9.4 'Here' documents

Sometimes it's necessary to create a file (often a script) from a running script. To this purpose, there are *here documents*. This is a mechanism of sending multiple lines as input to a command. The lines are delimited by two occurrences of an arbitrary string, its first occurrence marked by `<<`:

```
somecommand <<ASTRING
lines
lines
ASTRING
```

Typically, the command is `cat`, and you use this to generate a new document containing the value of some dynamically defined variables.

```
#!/bin/bash
# this file: makesay.sh

cat >say.sh <<EOF
#!/bin/bash
echo $1
EOF
chmod +x say.sh
```

Now running `makesay.sh foo` generates a file `say.sh` that contains `echo foo`.

Remark You can prevent expansion of variables by escaping the dollar sign; alternatively, writing

```
cat >newfile <<'EOF'
```

prevents all expansion.

Remark In addition to 'here documents', there are also here strings:

```
some_command <<<"some string"
```

is equivalent to

```
echo "some string" | some_command
```

9.5 Error handling

Scripts, like any other type of program, can fail with some runtime condition.

1. If there is no clear error message, you can at least rerun your script with a line

```
set -x
```

which echoes each command to the terminal before execution

2. Each script has a return code, contained in the variable `$?` which you can inspect.
3. It is also possible that some command in the script fails, but the script continues running. You can prevent this with

```
set -e
```

which makes the script abort if any command fails. The additional option

```
set -o pipefail
```

will catch errors in a pipeline.

Here is an idiom for being a little more informative about errors:

```
errcode=0
some_command || errcode=$?
if [ $errcode -ne 0 ] ; then
    echo "ERROR: some_command failed with code=$errcode"
    exit $errcode
fi
```

The crucial second line contains an ‘or’ condition: either *some_command* succeeds, or you set *errcode* to its exit code. This conjunction always succeeds, so now you can inspect the exit code.

10 Expansion

The shell performs various kinds of expansion on a command line, that is, replacing part of the commandline with different text.

10.1 Brace expansion

If you need to operate on a number of files with similar names, but which can not be describe with a wildcard, you can use brace expansion. In between braces you give the explicit list of variants in the names:

```
$ echo a{b,cc,ddd}e
abe acce addde
```

This can for instance be used to delete all extension of some base file name:

```
$ rm tmp.{c,s,o} # delete tmp.c tmp.s tmp.o
```

10.2 Tilde expansion

The tilde gives your own, or someone else's home directory:

```
$ echo ~  
/share/home/00434/eijkhout  
$ echo ~eijkhout  
/share/home/00434/eijkhout
```

10.3 Variable expansion

This gives the value of shell variables:

```
$ x=5  
$ echo $x  
5
```

Undefined variables do not give an error message:

```
$ echo $y
```

There are many variations on parameter expansion. Above you already saw that you can strip trailing characters:

```
$ a=b.c  
$ echo ${a%.c}  
b
```

Here is how you can deal with undefined variables:

```
$ echo ${y:-0}  
0
```

10.4 Command manipulation

10.4.1 History expansion

The shell keeps a history of your previously executed commands. The command `history` gives you the list, each command prefixed with its number in the command history. Let's say this is part of your history:

```
1669 make format  
1670 make clean  
1671 ls foo*  
1672 hostname
```

You can now re-execute previous commands, either exactly or with modifications, through a number of mechanisms.

- You can refer to a command by its number: `!1671` will execute `ls foo*` again.
- You can refer to a command by its initial characters: `!make` will execute the last `make` command, in this case `make clean`.

- Should you want to re-execute a command, but its initial characters are not unique, then you can refer to a unique non-initial substring: `!format?` will execute the last command containing `format`.

If you want to modify a previous commandline, retrieved by any of the above, there are several possibilities:

- You can display a previous command without executing it: `!make:p` will echo the command but not execute it. You can then use arrow-up to make it available for editing.
- If you mistype a command, say `make fromat`, you can easily edit it with `^fro^for^` to replace a substring by another string.
- Another way of editing a command is by a *sed*-like syntax: `!!:s/fro/for/`
- These mechanisms can be combined: if you are not sure whether an edit gave the right result, display the result as `!!:s/fro/for/:p`
- Finally, a global replace on a previous command is possible: `!make:s/a/b/:g&`

And there are more options, but these are a good basic set.

10.4.2 Command substitution

The backquote mechanism (section 5.3.3 above) is known as command substitution. It allows you to evaluate part of a command and use it as input for another. For example, if you want to ask what type of file the command `ls` is, do

```
$ file `which ls`
```

This first evaluates `which ls`, giving `/bin/ls`, and then evaluates `file /bin/ls`. As another example, here we backquote a whole pipeline, and do a test on the result:

```
$ echo 123 > w
$ cat w
123
$ wc -c w
  4 w
$ if [ `cat w | wc -c` -eq 4 ] ; then echo four ; fi
four
```

10.5 Arithmetic expansion

Unix shell programming is very much oriented towards text manipulation, but it is possible to do arithmetic. Arithmetic substitution tells the shell to treat the expansion of a parameter as a number:

```
$ x=1
$ echo ${x*2}
2
```

Integer ranges can be used as follows:

```
$ for i in {1..10} ; do echo $i ; done
1
2
3
```

```
4
5
6
7
8
9
10
```

(but see also the `seq` command in section 8.2.)

11 Startup files

In this tutorial you have seen several mechanisms for customizing the behavior of your shell. For instance, by setting the `PATH` variable you can extend the locations where the shell looks for executables. Other environment variables (section 7) you can introduce for your own purposes. Many of these customizations will need to apply to every session, so you can have *shell startup files* that will be read at the start of any session.

Popular things to do in a startup file are defining [aliases](#):

```
alias grep='grep -i'
alias ls='ls -F'
```

and setting a custom commandline *prompt*.

The name of the startup file depends on your shell: `.bashrc` for Bash, `.cshrc` for the C-shell, and `.zshrc` for the Z-shell. These files are read everytime you log in (see below for details), but you can also [source](#) them directly:

```
source ~/.bashrc
```

You would do this, for instance, if you have edited your startup file.

Unfortunately, there are several startup files, and which one gets read is a complicated functions of circumstances. Here is a good common sense guideline²:

- Have a `.profile` that does nothing but read the `.bashrc`:

```
# ~/.profile
if [ -f ~/.bashrc ]; then
    source ~/.bashrc
fi
```

- Your `.bashrc` does the actual customizations:

```
# ~/.bashrc
# make sure your path is updated
if [ -z "$MYPATH" ]; then
    export MYPATH=1
    export PATH=$HOME/bin:$PATH
fi
```

2. Many thanks to Robert McLay for figuring this out.

12 Shell interaction

Interactive use of Unix, in contrast to script writing (section 9), is a complicated conversation between the user and the shell. You, the user, type a line, hit return, and the shell tries to interpret it. There are several cases.

- Your line contains one full command, such as `ls foo`: the shell will execute this command.
- You can put more than one command on a line, separated by semicolons: `mkdir foo; cd foo`. The shell will execute these commands in sequence.
- Your input line is not a full command, for instance `while [ 1]`. The shell will recognize that there is more to come, and use a different prompt to show you that it is waiting for the remainder of the command.
- Your input line would be a legitimate command, but you want to type more on a second line. In that case you can end your input line with a backslash character, and the shell will recognize that it needs to hold off on executing your command. In effect, the backslash will hide (*escape*) the return.

When the shell has collected a command line to execute, by using one or more of your input line or only part of one, as described just now, it will apply expansion to the command line (section 10). It will then interpret the commandline as a command and arguments, and proceed to invoke that command with the arguments as found.

There are some subtleties here. If you type `ls *.c`, then the shell will recognize the wildcard character and expand it to a command line, for instance `ls foo.c bar.c`. Then it will invoke the `ls` command with the argument list `foo.c bar.c`. Note that `ls` does not receive `*.c` as argument! In cases where you do want the unix command to receive an argument with a wildcard, you need to escape it so that the shell will not expand it. For instance, `find . -name \*.c` will make the shell invoke `find` with arguments `. -name *.c`.

13 The system and other users

13.1 System information

Most of the above material holds for any Unix or Linux system. Sometimes you need to know detailed information about the system. The following tell you something about what's going on:

which processes are running on the system; use `top -u` to get this sorted the amount of cpu time they are currently taking. (On Linux, try also the `vmstat` command.)
how long has it been since your last reboot?

Sometimes you want to know what system you are actually using. Usually you can get some information out of `uname`:

```
$ uname -a
Linux staff.frontera.tacc.utexas.edu 3.10.0-1160.45.1.el7.x86_64 #1 SMP Wed Oct 13 17:20:51 UTC
2021 x86_64 x86_64 x86_64 GNU/Linux
```

This still doesn't tell you what *Linux distribution* you are on. For that, some of the following may work:


```
$ lsb_release -a
LSB Version:      :core-4.1-amd64:core-4.1-noarch:cxx-4.1-amd64:cxx-4.1-noarch:desktop-4.1-amd64:
                  desktop-4.1-noarch:languages-4.1-amd64:languages-4.1-noarch:printing-4.1-amd64:printing-4.1-
                  noarch
Distributor ID: CentOS
Description:      CentOS Linux release 7.9.2009 (Core)
Release:          7.9.2009
Codename:         Core
```

or

```
$ ls /etc/*release
/etc/centos-release /etc/os-release@ /etc/redhat-release@ /etc/system-release@
$ cat /etc/*release
CentOS Linux release 7.9.2009 (Core)
NAME="CentOS Linux"
VERSION="7 (Core)"
ID="centos"
ID_LIKE="rhel fedora"
VERSION_ID="7"
PRETTY_NAME="CentOS Linux 7 (Core)"
ANSI_COLOR="0;31"
CPE_NAME="cpe:/o:centos:centos:7"
HOME_URL="https://www.centos.org/"
BUG_REPORT_URL="https://bugs.centos.org/"

CENTOS_MANTISBT_PROJECT="CentOS-7"
CENTOS_MANTISBT_PROJECT_VERSION="7"
REDHAT_SUPPORT_PRODUCT="centos"
REDHAT_SUPPORT_PRODUCT_VERSION="7"

CentOS Linux release 7.9.2009 (Core)
CentOS Linux release 7.9.2009 (Core)
```

13.2 Users

Unix is a multi-user operating system. Thus, even if you use it on your own personal machine, you are a user with an *account* and you may occasionally have to type in your username and password.

If you are on your personal machine, you may be the only user logged in. On university machines or other servers, there will often be other users. Here are some commands relating to them.

show your login name.

show the other users currently logged in.

get information about another user; you can specify a user's login name here, or their real name, or other identifying information the system knows about.

13.3 Groups and ownership

In section 2.3 you saw that there is a permissions category for 'group'. This allows you to open up files to your close collaborators, while leaving them protected from the wide world.

When your account is created, your system administrator will have assigned you to one or more groups. (If you admin your own machine, you'll be in some default group; read on for adding yourself to more groups.)

The command `groups` tells you all the groups you are in, and `ls -l` tells you what group a file belongs to. Analogous to `chmod`, you can use `chgrp` to change the group to which a file belongs, to share it with a user who is also in that group.

Creating a new group, or adding a user to a group needs system privileges. To create a group:

```
sudo groupadd new_group_name
```

To add a user to a group:

```
sudo usermod -a -G thegroup theuser
```

While you can change the group of a file, at least between groups that you belong to, changing the owning user of a file with `chown` needs root privileges. See section 13.4.

13.4 The super user

Even if you own your machine, there are good reasons to work as much as possible from a regular user account, and use *root privileges* only when strictly needed. (The root account is also known as the *super user*.) If you have root privileges, you can also use that to 'become another user' and do things with their privileges, with the `sudo` ('superuser do') command.

`uptime whoami who finger otheruser` To execute a command as another user:

```
sudo -u otheruser command arguments
```

- To execute a command as the root user:

```
sudo command arguments
```

- Become another user:

```
sudo su - otheruser
```

- Become the *super user*:

```
sudo su -
```

Change the owning user of a file is done with `chown`:

```
sudo chown somefile someuser
sudo chown -R somedir someuser
```

14 Connecting to other machines: ssh and scp

No man is an island, and no computer is either. Sometimes you want to use one computer, for instance your laptop, to connect to another, for instance a supercomputer.

If you are already on a Unix computer, you can log into another with the 'secure shell' command `ssh`, a more secure variant of the old 'remote shell' command `rsh`:

```
ssh yourname@othermachine.otheruniversity.edu
```

where the yourname can be omitted if you have the same name on both machines.

To only copy a file from one machine to another you can use the ‘secure copy’ *scp*, a secure variant of ‘remote copy’ *rcp*. The *scp* command is much like *cp* in syntax, except that the source or destination can have a machine prefix.

To copy a file from the current machine to another, type:

```
scp localfile yourname@othercomputer:otherdirectory
```

where yourname can again be omitted, and otherdirectory can be an absolute path, or a path relative to your home directory:

```
# absolute path:
scp localfile yourname@othercomputer:/share/
# path relative to your home directory:
scp localfile yourname@othercomputer:mysubdirectory
```

Leaving the destination path empty puts the file in the remote home directory:

```
scp localfile yourname@othercomputer:
```

Note the colon at the end of this command: if you leave it out you get a local file with an ‘at’ in the name.

You can also copy a file from the remote machine. For instance, to copy a file, preserving the name:

```
scp yourname@othercomputer:otherdirectory/otherfile .
```

15 The sed and awk tools

Apart from fairly small utilities such as *tr* and *cut*, Unix has some more powerful tools. In this section you will see two tools for line-by-line transformations on text files. Of course this tutorial merely touches on the depth of these tools; for more information see [1, 2].

15.1 Stream editing with sed

Unix has various tools for processing text files on a line-by-line basis. The stream editor *sed* is one example. If you have used the *vi* editor, you are probably used to a syntax like *s/foo/bar/* for making changes. With *sed*, you can do this on the commandline. For instance

```
sed 's/foo/bar/' myfile > mynewfile
```

will apply the substitute command *s/foo/bar/* to every line of *myfile*. The output is shown on your screen so you should capture it in a new file; see section 6 for more on output *redirection*.

- If you have more than one edit, you can specify them with

```
sed -e 's/one/two/' -e 's/three/four/'
```

- If an edit needs to be done only on certain lines, you can specify that by prefixing the edit with the match string. For instance

```
sed '/^a/s/b/c/'
```

only applies the edit on lines that start with an a. (See section 3 for regular expressions.)

You can also apply it on a numbered line:

```
sed '25/s/foo/bar'
```

- The a and i commands are for ‘append’ and ‘insert’ respectively. They are somewhat strange in how they take their argument text: the command letter is followed by a backslash, with the insert/append text on the next line(s), delimited by the closing quote of the command.

```
sed -e '/here/a\  
appended text  
' -e '/there/i\  
inserted text  
' -i file
```

- Traditionally, sed could only function in a stream, so the output file always had to be different from the input. The GNU version, which is standard on Linux systems, has a flag -i which edits ‘in place’:

```
sed -e 's/ab/cd/' -e 's/ef/gh/' -i thefile
```

15.2 awk

The **awk** utility also operates on each line, but it can be described as having a memory. An awk program consists of a sequence of pairs, where each pair consists of a match string and an action. The simplest awk program is

```
cat somefile | awk '{ print }'
```

where the match string is omitted, meaning that all lines match, and the action is to print the line. Awk breaks each line into fields separated by whitespace. A common application of awk is to print a certain field:

```
awk '{print $2}' file
```

prints the second field of each line.

Suppose you want to print all subroutines in a Fortran program; this can be accomplished with

```
awk '/subroutine/ {print}' yourfile.f
```

Exercise 3.43. Build a command pipeline that prints of each subroutine header only the subroutine name. For this you first use sed to replace the parentheses by spaces, then awk to print the subroutine name field.

Awk has variables with which it can remember things. For instance, instead of just printing the second field of every line, you can make a list of them and print that later:

```
cat myfile | awk 'BEGIN {v="Fields:"} {v=v " " $2} END {print v}'
```

As another example of the use of variables, here is how you would print all lines in between a BEGIN and END line:

```
cat myfile | awk '/END/ {p=0} p==1 {print} /BEGIN/ {p=1} '
```

Exercise 3.44. The placement of the match with BEGIN and END may seem strange. Rearrange the awk program, test it out, and explain the results you get.

16 Review questions

Exercise 3.45. Devise a pipeline that counts how many users are logged onto the system, whose name starts with a vowel and ends with a consonant.

Exercise 3.46. Pretend that you're a professor writing a script for homework submission: if a student invokes this script it copies the student file to some standard location.

```
submit_homework myfile.txt
```

For simplicity, we simulate this by making a directory `submissions` and two different files `student1.txt` and `student2.txt`. After

```
submit_homework student1.txt
submit_homework student2.txt
```

there should be copies of both files in the `submissions` directory. Start by writing a simple script; it should give a helpful message if you use it the wrong way.

Try to detect if a student is cheating. Explore the `diff` command to see if the submitted file is identical to something already submitted: loop over all submitted files and

1. First print out all differences.
2. Count the differences.
3. Test if this count is zero.

Now refine your test by catching if the cheating student randomly inserted some spaces.

For a harder test: try to detect whether the cheating student inserted newlines. This can not be done with `diff`, but you could try `tr` to remove the newlines.

References

- [1] Alfred V. Aho, Brian W. Kernighan, and Peter J. Weinberger. *The Awk Programming Language*. Addison-Wesley Series in Computer Science. Addison-Wesley Publ., 1988. ISBN 020107981X, 9780201079814. [Cited on page 35.]
- [2] Dale Dougherty and Arnold Robbins. *sed & awk*. O'Reilly Media, 2nd edition edition. Print ISBN: 978-1-56592-225-9, ISBN 10:1-56592-225-5; Ebook ISBN: 978-1-4493-8700-6, ISBN 10:1-4493-8700-4. [Cited on page 35.]